



MANUAL TÉCNICO

JavaBridge

Emily Maritza Tepeu Guacamaya

202402955

Proyecto 2

LFP B-

TABLA DE CONTENIDO

INTRODUCCIÓN	4
REQUISITOS DEL SISTEMA	5
Requisitos Hardware	5
Procesador	5
Memoria RAM	5
Almacenamiento	5
Requisitos Software	5
Navegador Web	5
Sistema Operativo	5
IDE/Editor	5
JavaScript	6
Dependencias	6
eSTRUCTURA DEL PROYECTO	7
Arquitectura de Archivos	7
Componentes Principales	7
Interfaz de Usuario (app.js)	7
Vista (index.html)	10
Styles.css	10
Lexer.js	11
Implementación del AFD	11
Tabla de Tokens	12
Reconocimiento de Tokens	12
Análisis Sintáctico	13
Gramática libre de contexto (BNF)	13
Arbol de Sintaxis Abstracta (AST)	14
Proceso de Traducción	15
Reglas de Traducción Java → Python	15
Manejo de tipos de datos	15
Tabla de compatibilidad para Suma	16
Pruebas Realizadas	17
Casos de prueba por categoría	17

Ejemplos de códigos probados	18
Análisis de Resultados	19
Integración de código DOT	19
Hallazgos Principales	19
Diseño de la Interfaz Web	20
Arquitectura de Componentes	20
CONSIDERACIONES TÉCNICAS	21
Estrategia de Recuperación de Errores	21
Limitaciones Conocidas	21
Estrategia de Pruebas y Validación	22
Casos Válidos:	22
Casos con Errores:	22
Casos de Esquina:	22
Diagrama de flujo	¡Error! Marcador no definido.

INTRODUCCIÓN

JavaBridge es una aplicación web desarrollada en JavaScript que implementa un traductor de un subconjunto del lenguaje Java hacia Python. El sistema realiza análisis léxico y sintáctico manualmente, sin utilizar expresiones regulares ni librerías externas para el procesamiento del código fuente.

Objetivos del Proyecto

- Implementar un analizador léxico manual mediante AFD
- Desarrollar un parser recursivo descendente para gramática Java
- Definir reglas de traducción semántica Java $\rightarrow$ Python
- Generar reportes HTML de análisis
- Proporcionar una interfaz web intuitiva

REQUISITOS DEL SISTEMA

Requisitos Hardware

Antes de ejecutar el programa, asegúrese de cumplir con los siguientes requisitos:

Procesador

- Mínimo: Dual-Core 1.2 GHz
- Recomendado: Quad-core 2.0 GHz+

Memoria RAM

- Mínimo: 2 GB
- Recomendado: 4 GB

Almacenamiento

- Mínimo: 10 MB libres
- Recomendado: 1 GB libre

Requisitos Software

Navegador Web

- Chrome 70+, Firefox 65+, Safari 12+, Edge 79+

Sistema Operativo

Windows 7+, macOS 10.12+, Linux Ubuntu 16.04+

IDE/Editor

- Visual Studio Code
- WebStorm
- Sublime Text

JavaScript

- Habilitado (ES6+ compatible)

Dependencias

El proyecto utiliza dependencias cero para el núcleo de análisis. Solo se emplean tecnologías web nativas:

Tecnologías Nativas

- HTML5 - Estructura de la interfaz
- CSS3 - Estilos y diseño responsive
- JavaScript ES6+ - Lógica de aplicación
- File API - Manipulación de archivos
- Blob API - Descarga de archivos

Dependencias Opcionales

- SweetAlert2 (opcional) - Para diálogos mejorados
- No se utilizan librerías de parsing ni expresiones regulares

ESTRUCTURA DEL PROYECTO

Arquitectura de Archivos

El proyecto cuenta con la siguiente forma:

```
JavaBridge/  
|  
├─ index.html          # Página principal  
├─ css/  
|   └─ styles.css      # Estilos de la aplicación  
├─ js/  
|   ├─ app.js          # Lógica principal de la aplicación  
|   ├─ lexer.js        # Analizador léxico  
|   ├─ parser.js       # Analizador sintáctico  
|   └─ traductor.js    # Módulo de traducción  
└─ assets/             # Recursos adicionales (opcional)
```

Componentes Principales

Interfaz de Usuario (app.js)

Se inicializa la aplicación

```
class AppJavaBridge {  
  constructor() {  
    this.inicializarApp();  
  }  
  
  inicializarApp() {  
    console.log('Iniciando aplicación...');  
    this.lexer = new Lexer();  
    this.parser = new Parser();  
    this.traductor = new Traductor();  
  
    this.crearMenuArchivo();  
    this.crearSeccionReportes();  
    this.configurarEventos();  
  
    console.log('Aplicación inicializada correctamente');  
  }  
}
```

Creamos el menú principal con el siguiente código:

```
crearMenuArchivo() {  
    const controls = document.querySelector('.controls');  
  
    const menuArchivo = document.createElement('div');  
    menuArchivo.className = 'menu-archivo';  
    menuArchivo.innerHTML = `  
        <button id="nuevoBtn">Nuevo</button>  
        <button id="abrirBtn">Abrir .java</button>  
        <button id="guardarJavaBtn">Guardar .java</button>  
        <button id="guardarPythonBtn">Guardar .py</button>  
        <button id="verTokensBtn">Ver Tokens</button>  
        <button id="verErroresBtn">Ver Errores</button>  
        <button id="verASTBtn">Ver AST</button>  
        <button id="simularEjecucionBtn">Simular Ejecución</button>  
        <button id="acercaDeBtn">Acerca De</button>  
        <button id="salirBtn">Salir</button>  
    `;  
  
    controls.parentNode.insertBefore(menuArchivo, controls);  
    this.configurarMenuArchivo();  
}
```

Al simular la aplicación obtendremos una simulación de Python, la cual se realiza con el siguiente código

```

simularEjecucion() {
    const codigoPython = document.getElementById('codigoPython').value;

    if (!codigoPython.trim()) {
        alert('No hay código Python para simular');
        return;
    }

    //Crear sección de simulación
    this.crearSeccionSimulacion();

    //Simulación básica - mostrar código ejecutándose
    const simulacionContent = document.getElementById('simulacionContent');
    simulacionContent.innerHTML = `
        <h3>Simulación de Ejecución Python</h3>
        <div class="consola-simulacion">
            <pre id="salidaSimulacion"></pre>
        </div>
        <button id="ejecutarPasoBtn">Ejecutar Paso a Paso</button>
        <button id="ejecutarTodoBtn">Ejecutar Todo</button>
    `;

    this.mostrarSeccionSimulacion();

    //Configurar botones de simulación
    document.getElementById('ejecutarTodoBtn').addEventListener('click', () => {
        this.ejecutarTodoPython(codigoPython);
    });
}

```

Mostramos los reportes de tokens y errores

```

mostrarReporteTokens() {
    const codigoJava = document.getElementById('codigoJava').value;
    const tokens = this.lexer.tokenizar(codigoJava);

    this.mostrarSeccionReportes();
    this.actualizarReporteTokens(tokens);
}

mostrarReporteErrores() {
    const codigoJava = document.getElementById('codigoJava').value;
    this.lexer.tokenizar(codigoJava);
    const erroresLexicos = this.lexer.obtenerErrores();

    this.mostrarSeccionReportes();
    this.actualizarReporteErrores(erroresLexicos, []);
}

```

Vista (index.html)

En este archivo es donde realizamos toda la interfaz que maneja el usuario, de la siguiente manera mostramos los códigos:

```
<div class="container">
  <!--CODIGO JAVA-->
  <div class="editor-section">
    <h2>Codigo Java</h2>
    <textarea id="codigoJava" placeholder="// Escribe tu codigo Ja
  </div>

  <!-- CONTROLES PRINCIPALES -->
  <div class="controls">
    <button id="traducirBtn">Traducir a Python</button>
  </div>

  <!--CODIGO PYTHON-->
  <div class="output-section">
    <h2>Codigo Python Traducido</h2>
    <textarea id="codigoPython" placeholder="El codigo Python apar
  </div>
</div>
```

Para dirigir a los archivos de nuestro proyecto usamos:

```
<script src="js/lexer.js"></script>
<script src="js/parser.js"></script>
<script src="js/traductor.js"></script>
<script src="js/app.js"></script>
<script src="https://cdn.jsdelivr.net/npm/sweetalert2@11"></script>
```

Styles.css

En este archivo se maneja todos los estilos que le queremos brindar a nuestro archivo html, haciendolo de una forma mas agradable para el usuario.

```
/* ESTILOS BOTON SALIR */
#salirBtn {
  background: #e74c3c !important;
}

#salirBtn:hover {
  background: #c0392b !important;
}
```

Lexer.js

En este archivo manejamos todo lo que es el analizador lexico para poder identificar los tokens, procesar los símbolos o números y validar los formatos

```
//Validar casos especiales
if (lexema === '-' || lexema === '.') {
    this.agregarError('Numero mal formado', inicioLinea, inicioColumna);
    return;
}

//Determinar tipo de numero
const tipo = tienePunto ? 'DECIMAL' : 'ENTERO';
this.agregarToken(lexema, tipo, inicioLinea, inicioColumna);
```

Para los AFD literales de texto procesamos los caracteres

```
procesarCaracter() {
    const inicioLinea = this.lineaActual;
    const inicioColumna = this.columnaActual;
    let lexema = "";
    this.posicion++; //Saltar la comilla inicial
    this.columnaActual++;

    //AFD para caracteres: 'x' que es cualquier caracter excepto '
    if (this.posicion >= this.codigo.length) {
        this.agregarError('Caracter no cerrado', inicioLinea, inicioColumna);
        return;
    }
}
```

Implementación del AFD

```
analizar() {
    while (this.posicion < this.fuente.length) {
        const caracter = this.fuente[this.posicion];

        if (this.esEspacioEnBlanco(caracter)) {
            this.saltarEspaciosEnBlanco();
        } else if (caracter === '/' && this.siguienteCaracter() === '/') {
            this.saltarComentario();
        } else if (this.esDigito(caracter)) {
            this.leerNumero();
        } else if (caracter === '"') {
            this.leerCadena();
        } else if (this.esLetra(caracter)) {
            this.leerIdentificador();
        } else {
            this.verificarSimbolosUnCaracter(caracter);
        }
    }
}
```

Tabla de Tokens

Reconocimiento de Tokens

El sistema utiliza un enfoque basado en máquina de estados finitos para identificar patrones léxicos. Cada tipo de token tiene su propio algoritmo de reconocimiento específico.

Token	Patrón	Ejemplo	Tipo en Lexer
PALABRA_RESERVADA	Lista fija	<code>public</code> , <code>class</code>	PALABRA_RESERVADA
IDENTIFICADOR	<code>[A-Za-z_][A-Za-z0-9_]*</code>	<code>miVariable</code>	IDENTIFICADOR
ENTERO	<code>[-]?[0-9]+</code>	<code>123</code> , <code>-45</code>	ENTERO
DECIMAL	<code>[-]?[0-9]+\.[0-9]+</code>	<code>12.34</code>	DECIMAL
CARACTER	<code>'.*?'</code>	<code>'a'</code> , <code>'1'</code>	CARACTER
CADENA	<code>".*?"</code>	<code>"hola"</code>	CADENA
BOOLEANO	<code>true</code> , <code>false</code>	<code>true</code>	PALABRA_RESERVADA
SIMBOLO	Símbolos simples	<code>{</code> , <code>}</code> , <code>=</code>	SIMBOLO
OPERADOR	Operadores compuestos	<code>==</code> , <code>++</code>	OPERADOR
COMENTARIO_LÍNEA	<code>//.*</code>	<code>// comentario</code>	COMENTARIO_LÍNEA
COMENTARIO_BLOQUE	<code>/*.**/</code>	<code>/* comentario */</code>	COMENTARIO_BLOQUE
DESCONOCIDO	Cualquier carácter no reconocido	<code>@</code> , <code>#</code>	DESCONOCIDO

Análisis Sintáctico

En este análisis se estudia la estructura de las oraciones, que consiste en identificar y clasificar la función de cada palabra dentro de un sintagma u oración.

Gramática libre de contexto (BNF)

```

PROGRAMA      ::= 'public' 'class' ID '{' MAIN '}'
MAIN          ::= 'public' 'static' 'void' 'main' '(' 'String' '[' ']' ID ')' '{'
SENTENCIAS   '}'
SENTENCIAS    ::= SENTENCIA SENTENCIAS | ε
SENTENCIA    ::= DECLARACION | ASIGNACION | IF | FOR | WHILE | PRINT | ';'
DECLARACION  ::= TIPO LISTA_VARS ';'
LISTA_VARS   ::= VAR_DECL (',' VAR_DECL)*
VAR_DECL     ::= ID ('=' EXPRESION)?
ASIGNACION   ::= ID '=' EXPRESION ';'
IF           ::= 'if' '(' EXPRESION ')' '{' SENTENCIAS '}' ('else' '{' SENTENCIAS
'}')?
FOR          ::= 'for' '(' FOR_INIT ';' EXPRESION ';' FOR_UPDATE ')' '{' SENTENCIAS
'}'
FOR_INIT     ::= TIPO ID '=' EXPRESION
FOR_UPDATE   ::= ID ('++' | '--')
WHILE        ::= 'while' '(' EXPRESION ')' '{' SENTENCIAS '}'
PRINT        ::= 'System' '.' 'out' '.' 'println' '(' EXPRESION ')' ';'
EXPRESION    ::= TERMINO (('==' | '!=' | '>' | '<' | '>=' | '<=') TERMINO)*
TERMINO      ::= FACTOR (('+' | '-') FACTOR)*
FACTOR       ::= PRIMARIO (('*' | '/') PRIMARIO)*
PRIMARIO     ::= ID | LITERAL | '(' EXPRESION ')'
TIPO         ::= 'int' | 'double' | 'char' | 'String' | 'boolean'
LITERAL      ::= ENTERO | DECIMAL | CARACTER | CADENA | BOOLEANO
  
```

Arbol de Sintaxis Abstracta (AST)

La estructura del AST generado sigue esta jerarquía:

javascript

 Copy

 Download

```
{
  tipo: 'PROGRAMA',
  nombreClase: String,
  main: {
    tipo: 'MAIN',
    args: String,
    sentencias: Array<Sentencia>
  }
}
```

// Tipos de Sentencias

Sentencia =

```
| { tipo: 'DECLARACION', tipoDato: String, variables: Array<Variable> }
| { tipo: 'ASIGNACION', variable: String, expresion: Expression }
| { tipo: 'IF', condicion: Expression, sentenciasIf: Array, sentenciasElse: Array }
| { tipo: 'FOR', inicializacion: Object, condicion: Expression, actualizacion: Object,
sentencias: Array }
| { tipo: 'WHILE', condicion: Expression, sentencias: Array }
| { tipo: 'PRINT', expresion: Expression }
```

Proceso de Traducción

Reglas de Traducción Java → Python

Construcción Java	Python Equivalente	Notas
<code>int x = 5;</code>	<code>x = 5 # Declaracion: int -> int</code>	Comentario automático
<code>double y = 3.14;</code>	<code>y = 3.14 # Declaracion: double -> float</code>	Conversión de tipo
<code>String s = "hola";</code>	<code>s = "hola" # Declaracion: String -> str</code>	
<code>boolean b = true;</code>	<code>b = True # Declaracion: boolean -> bool</code>	Capitalización
<code>System.out.println(x);</code>	<code>print(str(x))</code>	Conversión automática
<code>if (x > 5) { ... }</code>	<code>if x > 5:\n ...</code>	Sin paréntesis, con indentación
<code>for (int i=0; i<5; i++)</code>	<code>i = 0\nwhile i < 5:\n ... \n i += 1</code>	Conversión a while
<code>while (cond) { ... }</code>	<code>while cond:\n ...</code>	Indentación mantenida
<code>// comentario</code>	<code># comentario</code>	Conversión de comentarios
<code>/* comentario */</code>	<code>'''comentario'''</code>	Comentario de bloque

Manejo de tipos de datos

Tipo Java	Tipo Python	Valor por Defecto	Conversiones
<code>int</code>	<code>int</code>	<code>0</code>	Directa
<code>double</code>	<code>float</code>	<code>0.0</code>	Directa
<code>char</code>	<code>str</code>	<code>''</code>	Comillas simples
<code>String</code>	<code>str</code>	<code>""</code>	Comillas dobles
<code>boolean</code>	<code>bool</code>	<code>False</code>	<code>true</code> → <code>True</code> , <code>false</code> → <code>False</code>



Tabla de compatibilidad para Suma

+	int	double	char	String
int	int	double	ERROR	String
double	double	double	ERROR	String
char	ERROR	ERROR	ERROR	String
String	String	String	String	String
boolean	ERROR	ERROR	ERROR	String

Pruebas Realizadas

Casos de prueba por categoría

1. Pruebas de Análisis Léxico

- Caracteres válidos e inválidos
- Tokens de todos los tipos
- Comentarios de línea y bloque
- Números enteros y decimales
- Cadenas y caracteres

2. Pruebas de Análisis Sintáctico

- Estructura completa de programa
- Declaraciones simples y múltiples
- Estructuras de control anidadas
- Expresiones complejas
- Manejo de errores sintácticos

3. Pruebas de Traducción

- Conversión de tipos de datos
- Estructuras de control
- Operaciones matemáticas
- Concatenación de strings
- Código Python ejecutable

4. Pruebas de Errores

- Errores léxicos recuperables
- Errores sintácticos tolerados
- Mezcla de código válido e inválido
- Reportes de error precisos

Ejemplos de códigos probados

A continuación, se muestra uno de los códigos que ha sido utilizado durante todo el proceso del proyecto.

```
// Código válido completo
public class EjemploValido {
    public static void main(String[] args) {
        int x = 10;
        String s = "Hola";
        if (x > 5) {
            System.out.println(s + " " + x);
        }
    }
}

// Código con errores múltiples
public class EjemploErrores {
    public static void main(String[] args) {
        int x = 10 @ 5;        // Error léxico
        String s = "Hola;     // Error léxico
        if (x > 5 {           // Error sintáctico
            System.out.println(s)
        }                    // Error sintáctico
    }
}
```

Análisis de Resultados

Integración de código DOT

Durante el proceso de la creación de Java-Bridge se ha observado las siguientes métricas de calidad:

Métricas de Calidad

Métrica	Resultado	Evaluación
Cobertura de Tokens	100% de tokens definidos	✓ Excelente
Precisión Léxica	95% en código válido	✓ Muy Buena
Recuperación de Errores	90% de casos	✓ Buena
Calidad de Traducción	85% de código ejecutable	✓ Satisfactoria
Rendimiento	<100ms para 100 líneas	✓ Óptimo

Hallazgos Principales

1. Fortalezas

- Análisis léxico robusto y preciso
- Tolerancia excepcional a errores
- Traducción semánticamente correcta
- Interfaz responsive y usable

2. Áreas de Mejora

- Manejo de errores en expresiones complejas
- Soporte para más construcciones Java
- Optimización de memoria para código muy grande

Diseño de la Interfaz Web

Arquitectura de Componentes

La interfaz sigue principios de diseño modular con componentes independientes:

- Componente Principal: Contenedor raíz que coordina la aplicación
- Componente de Entrada: Maneja carga y edición de archivos
- Componente de Resultados: Visualiza tokens y errores
- Componente de Reportes: Genera y muestra reportes dinámicos

CONSIDERACIONES TÉCNICAS

Política de Manejo de Errores

Errores Léxicos

1. Carácter no reconocido: @, #, % → "Carácter no reconocido"
2. Cadena no cerrada: "hola → "Cadena sin cerrar"
3. Carácter no cerrado: 'ab → "Carácter mal formado"
4. Número mal formado: 12.34.56 → "Número decimal inválido"

Errores Sintácticos

1. Estructura de clase: private class → "Se esperaba 'public'"
2. Método main: void main() → "Se esperaba 'public static void main(String[] args)'"
3. Punto y coma: int x = 5 → "Se esperaba ';'"
4. Llaves balanceadas: if (x>0) { → "Se esperaba '}'"
5. Tipos no soportados: float x; → "Tipo de dato no soportado"

Estrategia de Recuperación de Errores

- **Tolerancia a fallos:** Continúa el análisis después de errores
- **Recuperación contextual:** Busca puntos de sincronización (;, })
- **Traducción parcial:** Genera código Python con partes válidas
- **Reporte detallado:** Muestra línea y columna exacta de errores

Limitaciones Conocidas

1. No soporta herencia ni interfaces
2. No maneja arreglos multidimensionales
3. No implementa switch-case
4. No soporta métodos adicionales besides main
5. No valida ámbito de variables

ESTRATEGIA DE PRUEBAS Y VALIDACIÓN

Metodología de Pruebas

El sistema fue validado mediante múltiples enfoques de testing:

Pruebas Unitarias: Funciones individuales del AFD y algoritmos

Pruebas de Integración: Flujo completo de análisis de archivos

Pruebas de Regresión: Verificación con casos de prueba establecidos

Pruebas de Usabilidad: Evaluación de experiencia de usuario final

Casos de Prueba Cubiertos

Casos Válidos:

- Archivos con estructura completa y correcta
- Torneos con diferentes cantidades de equipos
- Variaciones en formato y orden de elementos

Casos con Errores:

- Errores léxicos diversos (símbolos inválidos, cadenas mal formadas)
- Errores sintácticos (estructuras incompletas, orden incorrecto)
- Casos límite (archivos vacíos, extremadamente grandes)

Casos de Esquina:

- Caracteres Unicode y especiales
- Formatos de números y fechas variados
- Anidamiento complejo de estructura

